

Music Import Dashboard Project Documentation

Project Overview

This project is a **local music ingestion and streaming asset pipeline** built on:

- **PHP (XAMPP / localhost)** → Dashboard + queue worker
- **Python** → Metadata extraction + thumbnail extraction + DB ingestion
- **yt-dlp** → Download YouTube audio / playlists
- **MySQL** → Store music metadata
- **FFmpeg** → Audio processing / embedded artwork extraction

It allows importing:

- Single YouTube songs
- Full YouTube playlists

through a web dashboard.

Folder Structure

```
App/  
├─ add_music.php  
├─ process_queue.php  
├─ status_api.php  
├─ worker.bat  
├─ queue.json  
├─ status.json  
├─ ingest_music_from_folder.py  
├─ input_music/  
├─ storage/  
│   └─ music/  
│       └─ thumbnails/
```

File Responsibilities

`add_music.php`

Frontend dashboard.

Responsibilities:

- Accept YouTube URL input
 - Add URL into queue
 - Show:
 - Queue
 - Under Process
 - Completed
 - Auto-refresh status using AJAX
-

queue.json

Stores pending jobs.

Example:

```
[  
  "https://youtu.be/...",  
  "https://youtube.com/playlist?..."  
]
```

Worker processes sequentially.

status.json

Tracks live execution state.

Structure:

```
{  
  "queue": [],  
  "processing": null,  
  "completed": []  
}
```

Used by dashboard for real-time updates.

worker.bat

Background infinite worker loop.

Runs:

```
php process_queue.php
```

every cycle.

Must stay open.

```
process_queue.php
```

Backend orchestrator.

Pipeline:

Step 1 — Pick next URL

Reads first queued item.

Step 2 — Download audio

Runs:

```
yt-dlp -x --audio-format mp3 --embed-metadata --embed-thumbnail
```

Output:

```
input_music/*.mp3
```

Step 3 — Run Python ingest

Executes:

```
python ingest_music_from_folder.py
```

Live stdout/stderr streamed to dashboard.

Step 4 — Success / Failure logging

Writes to completed jobs.

Green = success

Red = failure

```
ingest_music_from_folder.py
```

Core ingestion processor.

Responsibilities:

Reads MP3 files from:

```
input_music/
```

Extracts metadata:

- title
 - artist
 - album
 - duration
 - hash
-

Extracts thumbnail

Writes:

```
storage/thumbnails/
```

Copies audio to permanent library

Writes:

```
storage/music/
```

Inserts database record

Stores relative paths only:

```
storage/music/file.mp3  
storage/thumbnails/file.jpg
```

NOT absolute Windows paths.

Database Flow

Insert includes:

- title
- artist
- album
- duration
- hash
- file_path
- thumbnail_path

Example:

```
storage/music/song.mp3  
storage/thumbnails/hash.jpg
```

These are browser-accessible URLs.

Dashboard States

Queue

Waiting jobs.

Under Process

Live worker logs.

Examples:

```
Downloading...
Extracting metadata...
Copying music...
Inserting DB row...
```

Completed (Success)

Green card.

Shows:

```
Imported successfully
```

Completed (Failure)

Red card.

Shows exact error:

Examples:

- yt-dlp failed
 - ffmpeg failed
 - DB insert failed
 - duplicate file
 - permission denied
 - Python crash
-

Required Software

Install:

Python

Add to PATH.

Packages

```
pip install mutagen mysql-connector-python pillow
```

yt-dlp

```
pip install yt-dlp
```

FFmpeg

Add ffmpeg/bin to PATH.

Verify:

```
ffmpeg -version
```

XAMPP

Run:

- Apache
 - MySQL
-

How To Run

Start worker

```
cd C:\xampp\htdocs\dashboard\App  
worker.bat
```

Keep open.

Open dashboard

```
http://localhost/dashboard/App/add_music.php
```

Paste:

- Song URL
- Playlist URL

Click:

```
Add To Queue
```

Worker processes automatically.

Common Errors

Unicode crash

Fix:

```
sys.stdout.reconfigure(encoding='utf-8', errors='replace')
```

DB connection fail

Check:

- MySQL running
 - DB credentials
-

yt-dlp fail

Update:

```
yt-dlp -U
```

ffmpeg missing

Add to PATH.

Final Architecture

```
Dashboard
  ↓
Queue JSON
  ↓
Worker BAT
  ↓
process_queue.php
  ↓
yt-dlp download
  ↓
Python ingest
  ↓
Extract metadata + thumbnail
  ↓
Copy to storage
  ↓
Insert DB
  ↓
Dashboard success/failure
```

Project Purpose

Personal streaming platform content ingestion automation.

Instead of manual:

1. Download song
2. Move files
3. Extract metadata
4. Create thumbnails
5. Insert DB rows

Everything is automated from one URL input.